Linux Commands Study Material

This sheet explains the basic Linux commands you listed.

Online Practice Site

Practice these commands here:

https://killercoda.com/playgrounds/scenario/ubuntu

1. `ls`

Purpose: Lists files and folders in the current directory.

Syntax:

```
ls
```

Example:

```
ls
```

Use it when you want to see what is inside your current folder.

2. `ls -a`

Purpose: Lists all files and folders, including hidden ones.

Syntax:

```
ls -a
```

Example:

```
ls -a
```

Note:

- Hidden files in Linux usually begin with a dot, like `.bashrc`.

3. `ls -s`

Purpose: Shows file sizes in blocks along with file and folder names.

Syntax:

```
ls -s
```

Example:

```
ls -s
```

Use it when you want a quick idea of how much space files are using.

4. `ls -R`

Purpose: Lists files and folders recursively, including subdirectories.

Syntax:

```
ls -R
```

Example:

```
ls -R
```

Use it when you want to see the full folder structure.

5. `mkdir dir`

Purpose: Creates a new directory named `dir`.

Syntax:

```
mkdir dir
```

Example:

```
mkdir notes
```

This creates a folder called `notes`.

6. `mkdir dir1 dir2 dir3`

Purpose: Creates multiple directories in one command.

Syntax:

```
mkdir dir1 dir2 dir3
```

Example:

```
mkdir html css js
```

This creates three folders: `html`, `css`, and `js`.

Note:

- You wrote `mkdir dir1 dir2 dir 3`. In Linux, a space separates names, so that would create `dir1`, `dir2`, `dir`, and `3`.
- If you want one folder named `dir 3`, use quotes:

```
mkdir "dir 3"
```

7. `clear`

Purpose: Clears the terminal screen.

Syntax:

```
clear
```

Example:

```
clear
```

It does not delete files or commands. It only clears what you see on the screen.

8. `ll`

Purpose: Usually shows files in long-list format.

Syntax:

```
ll
```

Common output includes:

- file permissions
- owner name
- file size
- modified date
- file name

Important:

- `ll` is not a standard Linux command in every system.
- In many Linux distributions, it is an alias for:

```
ls -l
```

If `ll` does not work, use:

```
ls -l
```

9. `cd dir`

Purpose: Changes the current directory to `dir`.

Syntax:

```
cd dir
```

Example:

```
cd notes
```

After this command, you will move into the `notes` folder.

10. `cd ..`

Purpose: Moves one level up to the parent directory.

Syntax:

```
cd ..
```

Example:

```
cd ..
```

If you are inside `/home/user/docs`, this command moves you to `/home/user`.

11. `touch file_name`

Purpose: Creates a new empty file.

Syntax:

```
touch file_name
```

Example:

```
touch notes.txt
```

If the file already exists, `touch` updates its timestamp.

12. `touch file1 file2`

Purpose: Creates multiple empty files in one command.

Syntax:

```
touch file1 file2
```

Example:

```
touch index.html style.css
```

This creates both files at the same time.

13. `sudo nano file_name`

Purpose: Opens a file in the `nano` text editor with administrator permission.

Syntax:

```
sudo nano file_name
```

Example:

```
sudo nano notes.txt
```

What it does:

- opens the file in the `nano` editor
- if the file does not exist, `nano` can create it
- `sudo` gives permission to edit protected files

Common keys in `nano`:

- `Ctrl + S` saves the file on some systems
- `Ctrl + X` exits the editor

Important:

- In many Linux systems, the usual save command in `nano` is `Ctrl + O`, then press `Enter`.
- `Ctrl + X` is commonly used to exit.

14. `cat file_name`

Purpose: Displays the contents of a file in the terminal.

Syntax:

```
cat file_name
```

Example:

```
cat notes.txt
```

Use it when you want to quickly read a file without opening an editor.

15. `mkdir -p dir1/dir2/dir3`

Purpose: Creates nested directories in one command.

Syntax:

```
mkdir -p dir1/dir2/dir3
```

Example:

```
mkdir -p project/src/components
```

What it does:

- creates `project`
- creates `src` inside `project`
- creates `components` inside `src`

Note:

- `-p` helps create parent directories automatically if they do not already exist.

16. `cd dir1/dir2/dir3`

Purpose: Moves directly into a nested directory path.

Syntax:

```
cd dir1/dir2/dir3
```

Example:

```
cd project/src/components
```

This moves you directly into the `components` folder.

17. `cd ../../..`

Purpose: Moves up three directory levels.

Syntax:

```
cd ../../..
```

Example:

```
cd ../../..
```

If you are inside `/home/user/project/src/components`, this command moves you back to `/home/user`.

18. `cd ~`

Purpose: Moves directly to the home directory.

Syntax:

```
cd ~
```

Example:

```
cd ~
```

The `~` symbol represents the home directory of the current user.

19. `cd`

Purpose: Also moves directly to the home directory.

Syntax:

```
cd
```

Example:

```
cd
```

This is a shortcut way to return to your home directory.

Quick Revision Table

Command	Use
`ls`	List files and folders
`ls -a`	List all files, including hidden files
`ls -s`	Show file sizes
`ls -R`	List files recursively
`mkdir dir`	Create one directory
`mkdir dir1 dir2 dir3`	Create multiple directories
`clear`	Clear terminal screen
`ll`	Long list view, often alias of `ls -l`
`cd dir`	Move into a directory
`cd ..`	Move to parent directory
`touch file_name`	Create one empty file
`touch file1 file2`	Create multiple empty files
`sudo nano file_name`	Open or edit a file in nano with sudo

Command	Use
`cat file_name`	Show file contents in terminal
`mkdir -p dir1/dir2/dir3`	Create nested directories
`cd dir1/dir2/dir3`	Move into a nested directory path
`cd ../../..`	Move up three directory levels
`cd ~`	Move to home directory
`cd`	Move to home directory

Practice Commands

Try these in order:

```
mkdir linux_practice
cd linux_practice
touch file1.txt file2.txt
mkdir dir1 dir2 dir3
mkdir -p project/src/components
cd project/src/components
cd ../../..
sudo nano file1.txt
cat file1.txt
ls
ls -a
ls -s
ls -R
cd dir1
cd ..
cd ~
cd
clear
```

Key Points to Remember

- `ls` is for listing contents.
- `mkdir` is for creating directories.
- `cd` is for changing directories.
- `cd ~` and `cd` both take you to the home directory.

- `touch` is for creating empty files or updating timestamps.
- `clear` only cleans the terminal display.
- `ll` may not work on every Linux system.